**DECODELABS**

**FULL STACK DEVELOPMENT INTERNSHIP**

**BATCH 2026**

---

# INTERNSHIP REPORT

ON

## FULL STACK WEB DEVELOPMENT

*(Frontend UI · REST APIs · MongoDB Database Integration)*

---

*Submitted in partial fulfilment of the requirements of the*

**DecodeLabs Full Stack Internship Program**


**Submitted By**

**SHIV PRATAP SINGH**

B.Tech — Computer Science & Engineering (2027 Batch)

Indian Institute of Information Technology (IIIT) Manipur

*GitHub: github.com/shiv-pratap04/Decodelab-Internship*


**Under the Guidance of**

**DecodeLabs Mentorship Team**

Full Stack Development Track

**Duration of Internship**

May 2026 — June 2026 | 3 Weeks | Project-Based Learning


**DECODELABS**

*www.decodelabs.tech*

**September 2026**

# DECLARATION

I, Shiv Pratap Singh, a student of B.Tech in Computer Science and Engineering at the Indian Institute of Information Technology (IIIT) Manipur (2027 Batch), hereby declare that the internship report titled “Full Stack Web Development — DecodeLabs Internship Report (Batch 2026)” is an original work prepared by me during my internship with DecodeLabs under the Full Stack Development track.

I further declare that the projects documented herein — namely Inkwell (Responsive Blog UI), User Management REST API, and Database Integration System (Users & Tasks with MongoDB) — were designed, implemented, tested, and documented by me as part of the internship milestones. All sources of information, libraries, and external references used have been duly acknowledged. This report has not been submitted, in part or in full, to any other university, institute, or organization for the award of any degree, diploma, or certificate.

I take full responsibility for the authenticity of the technical content, architecture descriptions, API specifications, and the advanced backend extensions described in this report.

Date: _____

Place: _____

Signature: _____

Name: Shiv Pratap Singh

Program: B.Tech CSE, IIIT Manipur

Batch: DecodeLabs Full Stack 2026

# CERTIFICATE

## TO WHOM IT MAY CONCERN

This is to certify that Mr. Shiv Pratap Singh, a student of B.Tech (Computer Science & Engineering) at the Indian Institute of Information Technology (IIIT) Manipur, has successfully completed the Full Stack Development Internship at DecodeLabs (Batch 2026).

During the internship period, the intern worked on outcome-driven project milestones covering responsive frontend engineering, RESTful backend API design with Node.js and Express.js, and production-oriented MongoDB database integration using Mongoose ODM. The work submitted includes three progressive projects with complete source code, documentation, and API test collections, along with advanced backend extensions described in this report.

The intern has demonstrated sincerity, technical competence, and adherence to professional software engineering practices throughout the program. The contents of this report are a bonafide record of the work carried out under the DecodeLabs mentorship framework.

Date: _____

**Mentor / Project Guide**

DecodeLabs Mentorship Team
Full Stack Track

**Program Coordinator**

DecodeLabs
www.decodelabs.tech

*(Official DecodeLabs completion certificate may be attached as Annexure-A upon issue.)*

# ACKNOWLEDGEMENT

I express my sincere gratitude to DecodeLabs for providing a structured, outcome-first internship opportunity in Full Stack Development. The mentor-guided checkpoints, realistic delivery expectations, and project-proof orientation helped me convert theoretical knowledge into working software artifacts suitable for a professional portfolio.

I am thankful to the DecodeLabs mentorship team for continuous feedback on frontend design systems, REST API conventions, validation strategies, and database modelling. Their emphasis on clean architecture, accessibility, and production-ready error handling significantly improved the quality of my submissions.

I also acknowledge the open-source communities behind Node.js, Express.js, MongoDB, Mongoose, and the broader web platform standards (HTML5, CSS3, ECMAScript), which formed the foundation of the technical stack used in this internship.

Finally, I thank my institution, IIIT Manipur, and my peers for academic support and encouragement during the internship period.

**Shiv Pratap Singh**
B.Tech CSE, IIIT Manipur
DecodeLabs Full Stack Intern — Batch 2026

# ABSTRACT / EXECUTIVE SUMMARY

**Organization: DecodeLabs | Domain: Full Stack Development | Batch: 2026**

This report documents the work completed during a three-week, project-based Full Stack Development internship at DecodeLabs. The internship followed a progressive learning path from frontend fundamentals to backend APIs and database-backed systems, culminating in a portfolio of three complete projects hosted at github.com/shiv-pratap04/Decodelab-Internship.

Week 1 focused on building Inkwell, a responsive blog/news interface using pure HTML5, CSS3, and Vanilla JavaScript — mobile-first design system, CSS Grid/Flexbox, sticky navigation, live search and category filtering, accessible modals, and toast notifications.

Week 2 delivered a production-style User Management REST API using Node.js and Express.js with in-memory storage, express-validator middleware, centralized error handling, CORS, environment-based configuration, and a Postman collection.

Week 3 upgraded the backend to a Database Integration System with MongoDB and Mongoose: one-to-many Users–Tasks relationship, schema validation, pagination, sorting, filtering, populate, cascade cleanup on user delete, and global Mongoose-aware error handling.

Beyond mandated milestones, this report documents advanced backend work designed on top of the repository — JWT authentication & RBAC, refresh-token rotation, rate limiting, structured logging, request ID tracing, Redis caching, API versioning, soft deletes, audit trails, OpenAPI documentation, and Docker readiness — without modifying original project trees.

***Keywords:*** *Full Stack Development, REST API, Node.js, Express.js, MongoDB, Mongoose, Responsive Design, DecodeLabs, Internship Report*

# TABLE OF CONTENTS

DECLARATION
CERTIFICATE
ACKNOWLEDGEMENT
ABSTRACT / EXECUTIVE SUMMARY
TABLE OF CONTENTS
LIST OF TABLES / FIGURES / ABBREVIATIONS
CHAPTER 1 — INTRODUCTION
1.1 Background of the Internship
1.2 About DecodeLabs
1.3 Objectives of the Internship
1.4 Scope of Work
1.5 Organization of the Report
CHAPTER 2 — PROBLEM STATEMENT AND METHODOLOGY
CHAPTER 3 — WEEK 1: INKWELL RESPONSIVE BLOG UI
CHAPTER 4 — WEEK 2: USER MANAGEMENT REST API
CHAPTER 5 — WEEK 3: DATABASE INTEGRATION SYSTEM
CHAPTER 6 — ADVANCED BACKEND WORK (EXTENSIONS)
6.1–6.6 Auth, Security, Observability, DevOps, Module Map
CHAPTER 7 — RESULTS, LEARNINGS AND ANALYSIS
CHAPTER 8 — CONCLUSION AND FUTURE SCOPE
REFERENCES
ANNEXURES

# LIST OF TABLES

Table 2.1 — Technology Stack Used During Internship
Table 2.2 — Repository Directory Map
Table 3.1 — Inkwell Design Tokens (Color Palette)
Table 3.2 — Responsive Breakpoints
Table 3.3 — DecodeLabs Frontend Pillars — Compliance Status
Table 4.1 — User Management API Endpoints
Table 4.2 — Validation Rules for POST /api/users
Table 5.1 — User Model Fields and Constraints
Table 5.2 — Task Model Fields and Constraints
Table 5.3 — Database Integration API Surface
Table 6.1 — Advanced Backend Modules (Designed Extensions)
Table 6.2 — Proposed Role–Permission Matrix
Table 7.1 — Internship Deliverables Summary

# LIST OF FIGURES

Figure 2.1 — Internship Learning Progression
Figure 3.1 — Inkwell Desktop Layout Wireframe
Figure 3.2 — Inkwell Mobile Layout Wireframe
Figure 4.1 — Express Layered Architecture
Figure 5.1 — ER-style Relationship: User (1) — (N) Task
Figure 5.2 — Request Lifecycle with Error Handler
Figure 6.1 — Advanced Backend Architecture Blueprint
Figure 6.2 — JWT Access + Refresh Token Flow

# LIST OF ABBREVIATIONS

| Abbreviation | Full Form |
|---|---|
| API | Application Programming Interface |
| REST | Representational State Transfer |
| CRUD | Create, Read, Update, Delete |
| JWT | JSON Web Token |
| RBAC | Role-Based Access Control |
| ODM | Object Data Modeling |
| CORS | Cross-Origin Resource Sharing |
| ARIA | Accessible Rich Internet Applications |
| OWASP | Open Worldwide Application Security Project |

| Abbreviation | Full Form |
| --- | --- |
| CI/CD | Continuous Integration / Continuous Delivery |
| OpenAPI | Open API Specification (Swagger) |
| DTO | Data Transfer Object |

# CHAPTER 1

# INTRODUCTION

## 1.1 Background of the Internship

Modern software products are almost universally delivered as full-stack systems: a client interface that communicates with networked services, which in turn persist state in databases. An effective internship in this domain must train the intern not only in syntax of individual languages, but in the end-to-end reasoning required to design usable interfaces, stable contracts between client and server, and durable data models.

DecodeLabs structures its Full Stack Internship around visible project proof rather than passive content consumption. Interns complete milestone-based deliverables, receive mentor feedback, and publish portfolio-ready repositories. This report captures the complete journey of the Full Stack track for Batch 2026 as executed by the author.

## 1.2 About DecodeLabs

DecodeLabs is a technology learning and internship platform that operates on three pillars: (1) Outcome First — every internship is structured around visible project proof; (2) Mentor Guided — learners receive practical checkpoints, detailed feedback, and realistic delivery expectations; and (3) Career Aligned — portfolios and project presentation are tuned for recruiter evaluation. Official presence: www.decodelabs.tech.

## 1.3 Objectives of the Internship

**Broad Objective:** To gain hands-on competence in full-stack web development by designing, implementing, documenting, and testing progressive projects spanning frontend, backend, and database layers.

**Specific Objectives:**

1. Build a production-quality responsive UI using semantic HTML5, modern CSS, and Vanilla JS.
2. Design and implement a RESTful User Management API with validation and error handling.
3. Integrate MongoDB with Mongoose; model relationships; support pagination and filtering.
4. Apply software engineering practices: layered architecture, environment config, Postman tests.
5. Extend backend thinking toward production concerns: auth, security, observability, DevOps.
6. Document the entire body of work as a professional internship report and public repository.

## 1.4 Scope of Work

The scope includes three completed milestone projects (Week-1 Inkwell, Week-2 User Management REST API, Week-3 Database Integration System) as present in the internship repository, plus a dedicated chapter on advanced backend extensions that build upon — but do not alter — the original project source code.

## 1.5 Organization of the Report

- Chapter 1 introduces the internship, organization, and objectives.
- Chapter 2 states the problem, methodology, tools, and repository map.
- Chapter 3 details the Week-1 frontend project (Inkwell).
- Chapter 4 details the Week-2 REST API project.
- Chapter 5 details the Week-3 MongoDB integration project.
- Chapter 6 presents advanced backend work designed as extensions.
- Chapter 7 analyses results and learning outcomes.
- Chapter 8 concludes and outlines future scope.
- References and Annexures follow the main chapters.

# CHAPTER 2

# PROBLEM STATEMENT AND METHODOLOGY

## 2.1 Problem Identification

Entry-level full-stack competence is often fragmented: students may build static pages without accessibility, write Express servers without validation, or use MongoDB without modelling relationships and failure modes. Industry expects cohesive skills — readable UI systems, predictable HTTP contracts, schema integrity, and defensive error surfaces.

**Problem Statement:** Design and deliver a progressive set of full-stack artifacts that demonstrate (a) a responsive, accessible content UI; (b) a validated REST API with clean layering; and (c) a database-backed multi-resource system with relationships, pagination, and robust errors — then analyze how the backend can be hardened toward production readiness.

## 2.2 Expected Outcomes

- Outcome 1: Framework-free responsive blog UI meeting DecodeLabs visual and a11y pillars.
- Outcome 2: User CRUD API with express-validator, CORS, dotenv, and Postman coverage.
- Outcome 3: Users–Tasks system on MongoDB with populate, filters, and cascade cleanup.
- Outcome 4: Documented advanced backend blueprint (auth, security, ops) for future iteration.

## 2.3 Methodology and Tools

The internship followed an iterative, milestone-driven methodology: requirements briefing → design (UI wireframe / API contract / schema) → implementation → self-testing → documentation → mentor checkpoint. Version control was maintained via Git and GitHub.

**Table 2.1 — Technology Stack Used During Internship**

| Layer | Technologies |
|---|---|
| Frontend | HTML5, CSS3 (Grid, Flexbox, custom properties), Vanilla JavaScript (ES6+) |
| Fonts / Design | Google Fonts (Playfair Display, DM Sans), DecodeLabs 2025 mocha palette |
| Backend Runtime | Node.js (≥14) |
| Web Framework | Express.js (v4 / v5 across projects) |
| Validation | express-validator; Mongoose schema validators |
| Database | MongoDB Atlas; Mongoose ODM v8 |
| Cross-cutting | cors, dotenv, nodemon |
| API Testing | Postman collections |
| Tooling | Git, GitHub |
| Advanced (designed) | JWT, bcrypt, helmet, rate-limit, pino, Redis, Docker, OpenAPI |

**Figure 2.1 — Internship Learning Progression**

*[Week 1: UI Fundamentals] → [Week 2: REST + In-Memory] → [Week 3: MongoDB + Relationships] → [Advanced: Auth, Security, Ops]*

## 2.4 Repository Structure

All internship source code is organized week-wise in the public repository https://github.com/shiv-pratap04/Decodelab-Internship. This report is added under Internship-Report/ without modifying the original Week-* project trees.

**Table 2.2 — Repository Directory Map**

| Path | Contents |
|---|---|
| Week-1/Inkwell/ | Responsive Blog UI (HTML/CSS/JS + README) |
| Week-2/ | Express User API + Postman collection |
| Week-3/ | MongoDB Users & Tasks API + Postman |
| Internship-Report/ | This internship report (DOCX + MD + PDF) |

# CHAPTER 3

# WEEK 1: INKWELL — RESPONSIVE BLOG UI

## 3.1 Project Overview

Inkwell is a responsive blog/news interface built as Project 1 of the DecodeLabs Full Stack Internship. It deliberately uses zero frontend frameworks to force mastery of the web platform: semantic HTML5 landmarks, a CSS design system, and Vanilla JavaScript state management.

*Project path: Week-1/Inkwell/*

- index.html — Semantic structure and content
- style.css — Design system, layout, components, responsive rules (~1,150+ lines)
- app.js — Interactivity, filtering, modals, toasts (~340 lines)
- README.md — Feature matrix, design tokens, accessibility notes

## 3.2 Design System

A tokenized design system was implemented with CSS custom properties, aligned to the DecodeLabs 2025 UI/UX specification. Warm mocha neutrals are paired with soft blue accents for a grounded, editorial reading experience. Typography pairs Playfair Display (headings) with DM Sans (body/UI). Fluid type uses clamp(), e.g. font-size: clamp(1.75rem, 2.8vw, 2.6rem).

**Table 3.1 — Inkwell Design Tokens (Color Palette)**

| Token | Hex | Usage |
|---|---|---|
| --mocha | #A5856E | Primary brand, buttons, accents |
| --mocha-dark | #6b4f38 | Hover states, headings |
| --mocha-light | #f5ede6 | Backgrounds, tags |
| --blue | #A0D4E0 | Card gradients, highlights |
| --blue-dark | #2d7d8f | Blue avatar, links |
| --grey | #F2F0EA | Page background |
| --border | #e8e4dd | Borders |
| --text | #1a1714 | Body text, footer background |
| --text-muted | #7a7570 | Secondary text |

**Table 3.2 — Responsive Breakpoints (Mobile-First)**

| Breakpoint | Target | Key Behavior |
|---|---|---|
| Base | Mobile | Single-column flow, hamburger navigation |
| max-width: 1024px | Tablet | Sidebar stacks; grid adjusts |
| max-width: 768px | Mobile nav | Hide desktop nav; show hamburger |
| max-width: 480px | Small phones | Tighten padding; single column |
| >= 1024px | Desktop | Main 1fr + Sidebar 320px; sticky aside |

## 3.3 Layout Architecture

Macro layout uses CSS Grid; micro layout (nav rows, card meta, action clusters) uses Flexbox. Semantic landmarks: header, nav, main, section, article, aside, footer. Heading hierarchy: h1 (hero) → h2 (sections) → h3 (cards/widgets).

**Figure 3.1 — Desktop Layout Wireframe**

*[ Sticky Header / Nav ] → [ Hero — 2-col grid ] → [ Category pills ] → [ Articles (1fr) | Sidebar 320px ] → [ Footer — 4-col grid ]*

**Figure 3.2 — Mobile Layout Wireframe**

*[ Logo + hamburger ] → [ Slide-down menu + search ] → [ Hero stacked ] → [ Scrollable pills ] → [ Articles grid ] → [ Sidebar ] → [ Footer stacked ]*

## 3.4 Features Implemented

- Sticky header with scroll-aware shadow and backdrop blur
- Animated hamburger menu with aria-expanded state management
- Desktop + mobile search driving shared filter state
- Category pill filters combined with live text search (title + excerpt)
- Featured article card spanning full grid width on desktop
- Sidebar: trending list, tags cloud, authors, newsletter widget
- Subscribe modal with email validation, Escape/backdrop close
- Toast notifications with auto-dismiss (~2.8s) and aria-live
- Staggered entrance animations (fadeUp) and hero shimmer accents
- Keyboard activation of cards (Enter/Space) and Escape to close overlays

## 3.5 JavaScript Interactivity

app.js runs in strict mode with a small explicit state model (activeSearch, activeCat, toastTimer). Event listeners wire DOM controls to pure functions such as filterArticles(), which toggles card visibility by combining category and search predicates. No external state library was used — reinforcing fundamentals of DOM querying and accessible widget behavior.

## 3.6 Accessibility Audit

- Semantic HTML5 landmarks for assistive technology
- aria-label on icon buttons; aria-expanded on hamburger; aria-hidden on decorative nodes
- role=tablist / aria-selected on category tabs
- role=dialog + aria-modal on subscribe modal
- role=alert + aria-live on toast region
- :focus-visible rings on interactive cards
- Escape closes overlays; Enter/Space activates cards

**Table 3.3 — DecodeLabs Frontend Pillars — Compliance Status**

| Pillar | Status |
|---|---|
| Strategy & Blueprint (mobile-first, hierarchy-driven) | Met |

| Pillar | Status |
| --- | --- |
| Visual Design & 2025 Aesthetics (mocha, type pairing) | Met |
| Semantic HTML5 foundation | Met |
| CSS Grid + Flexbox framework | Met |
| JavaScript state & interactivity | Met |
| Audit — accessibility | Met |

# CHAPTER 4

# WEEK 2: USER MANAGEMENT REST API

## 4.1 Project Overview

Week 2 shifts focus from the browser to the server. The User Management REST API is a production-quality Express application that exposes user resources over HTTP using in-memory storage. The goal is to internalize REST conventions, middleware pipelines, validation, and consistent JSON error envelopes before introducing a real database.

*Project path: Week-2/ | Base URL: http://localhost:5000*
Tech stack: Node.js, Express.js, cors, dotenv, express-validator, nodemon.

## 4.2 Architecture and Layering

- server.js — Bootstrap, global middleware, 404 handler, error handler, process guards
- routes/userRoutes.js — HTTP method/path mapping to controllers
- controllers/userController.js — Request/response orchestration
- middleware/validateUser.js — express-validator rules + result gate
- data/users.js — In-memory repository (array + nextId counter)

**Figure 4.1 — Express Layered Architecture**

*Client → cors/json parser → Router (/api/users) → validateUser (on POST) → Controller → data/users repository → JSON | errors → centralized error middleware*

## 4.3 API Endpoints

**Table 4.1 — User Management API Endpoints**

| Method | Path | Status | Description |
|---|---|---|---|
| GET | /api/users | 200 | Return all users (array) |
| GET | /api/users/:id | 200 / 404 | Return single user by numeric id |
| POST | /api/users | 201 / 400 | Create user (validated body) |
| DELETE | /api/users/:id | 204 | Delete user (No Content) |

*Sample create response: { "success": true, "message": "User created", "data": { "id": 2, "name": "Shiv", "email": "shiv@gmail.com" } }*

## 4.4 Validation and Error Handling

**Table 4.2 — Validation Rules for POST /api/users**

| Field | Rules |
|---|---|
| name | required, trimmed, minimum 3 characters |
| email | required, trimmed, must be a valid email |

Validation failures return HTTP 400 with success:false and an errors array. Resource misses return 404. Unexpected failures map to HTTP 500 without leaking internal stack traces.

### 4.5 Testing with Postman

A complete Postman collection (User Management REST API.postman_collection.json) covers list, get-by-id, create (valid and invalid payloads), and delete flows. npm run dev starts nodemon; npm start runs the production entrypoint.

# CHAPTER 5

# WEEK 3: DATABASE INTEGRATION SYSTEM

## 5.1 Project Overview

Week 3 elevates the backend from an in-memory store to a production-oriented RESTful system backed by MongoDB. The Database Integration System manages Users and Tasks with a one-to-many relationship, schema validation, pagination, sorting, filtering, population of references, and centralized Mongoose-aware error handling.

*Project path: Week-3/ | Base URL: http://localhost:5000 | Health: GET /api/health*

## 5.2 Schema Design and Relationships

**Figure 5.1 — ER-style Relationship**

*User (1) ■■■■■■■■■■ (N) Task via Task.user → ObjectId ref User*

**Table 5.1 — User Model Fields and Constraints**

| Field | Type | Constraints |
|---|---|---|
| name | String | required, trim, min 2, max 50 |
| email | String | required, unique, lowercase, trim, regex email |
| age | Number | required, min 18, max 150 |
| timestamps | Date | createdAt / updatedAt |

**Table 5.2 — Task Model Fields and Constraints**

| Field | Type | Constraints |
|---|---|---|
| title | String | required, trim, min 3, max 100 |
| description | String | optional, trim, max 500 |
| completed | Boolean | default false |
| user | ObjectId | required, ref: User |
| timestamps | Date | createdAt / updatedAt |

## 5.3 User and Task CRUD Operations

**Table 5.3 — Database Integration API Surface**

| Method | Endpoint | Notes |
|---|---|---|
| POST | /api/users | Create user (name, email, age) |
| GET | /api/users | List users with page, limit, sort |
| GET | /api/users/:id | Fetch by ObjectId (format-checked) |
| PUT | /api/users/:id | Update with runValidators: true |
| DELETE | /api/users/:id | Delete user + Task.deleteMany |

| Method | Endpoint | Notes |
|---|---|---|
| GET | /api/users/:id/tasks | User profile joined with tasks |
| POST | /api/tasks | Create task; verify user; populate |
| GET | /api/tasks | Filter completed; sort; populate |
| GET | /api/tasks/:id | Fetch task with populated user |
| PUT | /api/tasks/:id | Update task; optional reassignment |
| DELETE | /api/tasks/:id | Delete task |
| GET | /api/health | Liveness probe |

ObjectId format is validated with a 24-hex regex before hitting the database (HTTP 400 on bad format). On user deletion, associated tasks are removed via Task.deleteMany to avoid orphans.

## 5.4 Pagination, Sorting and Filtering

- Users list: ?page=&limit=&sort= (default -createdAt); pagination metadata in response
- Tasks list: ?completed=true|false&sort=; filter object built dynamically
- Lean queries for read-heavy list endpoints
- populate("user") resolves foreign keys for task responses

## 5.5 Security and Performance Considerations

**Figure 5.2 — Request Lifecycle with Error Handler**

*HTTP → express.json → route → controller → Mongoose → success JSON | next(err) → errorHandler maps ValidationError / E11000 / CastError → clean JSON status*

- Schema-level validation + unique email index at the database layer
- Duplicate key (E11000) → readable “Email already exists”
- No sensitive internals in client-facing error messages
- CORS enabled; secrets via dotenv; fail-fast if MONGODB_URL missing
- Pagination limits payload size; stateless design for horizontal scale

# CHAPTER 6

# ADVANCED BACKEND WORK (EXTENSIONS)

## 6.1 Motivation for Advanced Work

The mandated Week-2 and Week-3 APIs correctly teach REST and persistence. Real backend engineering continues into identity, abuse resistance, operability, and safe evolution of contracts. Without changing the original Week-* source trees (preserving milestone integrity), advanced backend modules were designed as an extension blueprint beside the Database Integration System. This chapter records that advanced work in depth.

**Figure 6.1 — Advanced Backend Architecture Blueprint**

*Client → TLS → Reverse Proxy → Rate Limit & Helmet → AuthN (JWT) → AuthZ (RBAC) → Validation → Controllers → Services → Repositories → MongoDB; side paths: Redis Cache, Logger/Tracing, Audit Log*

## 6.2 Authentication & Authorization (JWT + RBAC)

The internship APIs are open by default. The advanced design introduces authentication so mutating operations require proof of identity, and authorization so privileges differ by role.

**Figure 6.2 — JWT Access + Refresh Token Flow**

*POST /auth/register → bcrypt (≥12) → store user | POST /auth/login → Access JWT (15m) + Refresh (7d, rotating) | Bearer access on requests | POST /auth/refresh rotates | POST /auth/logout revokes family*

**Table 6.2 — Proposed Role–Permission Matrix**

| Action | Guest | User | Admin |
|---|---|---|---|
| GET public health | Yes | Yes | Yes |
| Register / Login | Yes | Yes | Yes |
| CRUD own profile | No | Yes | Yes |
| CRUD own tasks | No | Yes | Yes |
| List all users | No | No | Yes |
| Delete any user/task | No | No | Yes |
| View audit logs | No | No | Yes |

- Access tokens carry sub, role, iat, exp; signed HS256 or RS256
- Passwords never stored in plaintext; bcrypt/argon2 only
- Refresh-token rotation mitigates replay if a token is stolen
- Optional email verification and password-reset one-time tokens
- Middleware chain: authenticate → authorize("admin") → controller

## 6.3 Security Hardening

Building on OWASP API Security guidance, the following controls were designed:

1. HTTP security headers via helmet (CSP, frameguard, HSTS in production)

2. Rate limiting (express-rate-limit / Redis-backed) on /auth/login and global API budget
3. Brute-force lockout counters per account/IP with exponential backoff
4. Strict CORS origin allowlists instead of fully open cors() in production
5. Input sanitization and payload size limits (express.json({ limit: "100kb" }))
6. Parameterized queries only (Mongoose) — no string-concatenated queries
7. Prevent mass assignment via DTO / allowlist in update handlers
8. Soft-delete flags (deletedAt) for recoverable deletes on sensitive entities
9. Secrets management: .env locally; vault in deployment; never commit secrets
10. Dependency auditing (npm audit) and lockfile discipline

## 6.4 Observability, Caching and Performance

- Structured logging with pino/winston: level, message, requestId, userId, route, latencyMs
- Request ID middleware (x-request-id) for supportability
- Metrics: request rate, error rate, p95 latency; optional Prometheus
- Liveness (/api/health) vs readiness (/api/ready checking Mongo ping)
- Redis cache for hot reads with TTL and write-through invalidation
- Compound index on Task { user: 1, completed: 1, createdAt: -1 }
- lean() / projections; cursor pagination as scale-up path
- Graceful shutdown on SIGTERM; compression middleware

## 6.5 API Lifecycle, DevOps and Documentation

- API versioning via URI prefix /api/v1
- OpenAPI 3.0 + Swagger UI at /api/docs
- Consistent envelope: { success, data, meta?, error? }
- Jest + supertest integration tests for auth + CRUD
- CI pipeline: install → lint → test → build image
- Dockerfile multi-stage; docker-compose with api + mongo + redis
- Twelve-factor config; NODE_ENV-specific behaviors
- Audit trail collection (actor, action, resource, before/after, IP, UA)
- Optional background jobs: email dispatch, token cleanup

## 6.6 Proposed Advanced Module Map

The following module map is the concrete extra backend work layer designed against the existing Week-3 architecture — documented so reviewers can evaluate engineering depth without disturbing graded milestone folders.

**Table 6.1 — Advanced Backend Modules (Designed Extensions)**

| Module | Responsibility | Key Tech |
|---|---|---|
| auth/ | Register, login, refresh, logout, reset | JWT, bcrypt, cookies |
| middleware/auth.js | Verify access token; attach req.user | jsonwebtoken |
| middleware/rbac.js | Role/permission checks | Custom policy map |

| Module | Responsibility | Key Tech |
|---|---|---|
| middleware/rateLimit.js | Throttle abusive clients | rate-limit, Redis |
| middleware/requestId.js | Correlation IDs | crypto.randomUUID |
| services/ | Business logic decoupled from HTTP | Plain JS services |
| repositories/ | Data access wrapping Mongoose | Mongoose models |
| utils/logger.js | Structured logs | pino / winston |
| utils/cache.js | Get/set/invalidate helpers | ioredis |
| models/AuditLog.js | Immutable activity records | Mongoose |
| models/RefreshToken.js | Rotating refresh sessions | Mongoose / Redis |
| docs/openapi.yaml | Contract & examples | OpenAPI 3 |
| tests/ | Unit + integration suites | Jest, supertest |
| Dockerfile / compose | Reproducible runtime | Docker |

**Illustrative advanced endpoints (design):**

- POST /api/v1/auth/register | login | refresh | logout | forgot-password | reset-password
- GET/PATCH /api/v1/me — current user profile (self-service)
- Existing /api/users and /api/tasks under /api/v1 with auth guards
- GET /api/v1/admin/audit-logs — admin only
- GET /api/v1/ready — Mongo + Redis readiness

Service-layer example: TaskService.create(actor, dto) verifies ownership (unless admin), writes Task + AuditLog, and busts cache keys. Controllers become thin HTTP adapters — a hallmark of maintainable backends.

# CHAPTER 7

# RESULTS, LEARNINGS AND ANALYSIS

## 7.1 Deliverables Achieved

**Table 7.1 — Internship Deliverables Summary**

| Week | Deliverable | Evidence | Status |
|---|---|---|---|
| 1 | Inkwell Responsive Blog UI | Week-1/Inkwell/* | Completed |
| 2 | User Management REST API | Week-2/* + Postman | Completed |
| 3 | DB Integration (Users/Tasks) | Week-3/* + Postman | Completed |
| — | Internship Report | Internship-Report/* | Completed |
| — | Advanced Backend Blueprint | Chapter 6 of this report | Completed |

## 7.2 Technical Results

• Frontend: fully responsive editorial UI with design tokens, a11y roles, interactive filtering

• Backend (memory): validated User CRUD with clean middleware pipeline and process guards

• Backend (MongoDB): dual-resource API with relationships, pagination, populate, cascade delete

• Quality artifacts: README documentation per project; Postman collections

• Advanced design: end-to-end path from open learning API → authenticated, observable service

## 7.3 Challenges and Resolutions

• Challenge: Balancing visual polish with accessibility. Resolution: ARIA roles, focus-visible, keyboard paths from the start.

• Challenge: Consistent API errors across validators and Mongoose. Resolution: Dedicated error middleware.

• Challenge: Orphan tasks after user deletion. Resolution: Task.deleteMany({ user: id }).

• Challenge: Invalid ObjectIds. Resolution: 24-hex pre-check → HTTP 400.

• Challenge: Showing advanced skill without touching milestones. Resolution: Chapter 6 blueprint only.

## 7.4 Key Learning Outcomes

1. Mobile-first CSS architecture and design-system thinking without UI frameworks
2. Express middleware composition, routing hygiene, and environment-based configuration
3. Schema design, referential patterns in document databases, and ODM validation
4. API product mindset: pagination, sorting, filtering, health checks, Postman contracts
5. Production backend instincts: authn/authz, rate limits, logging, caching, containers
6. Professional documentation and repository organization for portfolio presentation

### 7.5 Self-Assessment

All mandatory DecodeLabs project pillars for the submitted weeks were met. The additional backend blueprint demonstrates readiness to extend learning projects into secure, operable services. Areas for continued growth include full automated test suites, implementing the auth module in code, and deploying to a cloud free tier with CI/CD.

# CHAPTER 8

# CONCLUSION AND FUTURE SCOPE

## 8.1 Conclusion

The DecodeLabs Full Stack Internship (Batch 2026) provided a rigorous, project-proof path from responsive UI craftsmanship to database-backed API engineering. Through Inkwell, the User Management REST API, and the Database Integration System, the intern practiced the complete vertical of modern web development using HTML/CSS/JS, Node.js, Express, MongoDB, and Mongoose — while adhering to accessibility, validation, and documentation standards.

By further analyzing and specifying advanced backend capabilities (JWT authentication, RBAC, rate limiting, observability, caching, OpenAPI, and Docker), the work transcends checklist completion and reflects an engineer's view of how learning APIs mature into production systems. The public repository and this report together form a coherent portfolio artifact.

## 8.2 Future Scope

- Implement the Chapter 6 auth and RBAC modules in a Week-4+ branch
- Add Jest + supertest coverage targeting ≥80% critical-path lines
- Connect Inkwell frontend to the Week-3 API for a true full-stack demo
- Introduce TypeScript for shared DTO types across client and server
- Deploy API to a container host with MongoDB Atlas and Redis
- Add real-time features (WebSockets) and file uploads to object storage
- Explore GraphQL or tRPC while keeping REST for public integrators

## 8.3 Closing Remark

Internship programs succeed when interns leave with both running software and clearer engineering judgment. This engagement achieved both. The author thanks DecodeLabs for the structured opportunity and remains committed to continuous improvement as a full-stack engineer.

# REFERENCES


[1] DecodeLabs. (2026). Official website and internship verification portal. https://www.decodelabs.tech/

[2] Express.js documentation. https://expressjs.com/

[3] MongoDB Manual. https://www.mongodb.com/docs/manual/

[4] Mongoose ODM documentation. https://mongoosejs.com/docs/

[5] Node.js documentation. https://nodejs.org/docs/

[6] MDN Web Docs — HTML, CSS, JavaScript, ARIA. https://developer.mozilla.org/

[7] express-validator documentation. https://express-validator.github.io/docs/

[8] OWASP API Security Top 10. https://owasp.org/www-project-api-security/

[9] JWT Introduction (jwt.io). https://jwt.io/introduction

[10] OpenAPI Specification. https://swagger.io/specification/

[11] Twelve-Factor App methodology. https://12factor.net/

[12] Postman Learning Center. https://learning.postman.com/

[13] W3C Web Content Accessibility Guidelines (WCAG). https://www.w3.org/WAI/standards-guidelines/wcag/

[14] Repository: shiv-pratap04/Decodelab-Internship. https://github.com/shiv-pratap04/Decodelab-Internship

# ANNEXURES

## Annexure A — Internship Completion Certificate

(Attach the official DecodeLabs completion certificate / verification screenshot here when issued. Verification portal: https://www.decodelabs.tech/verification)

## Annexure B — How to Run the Projects

### B.1 Week 1 — Inkwell

*npx serve "Week-1/Inkwell" OR python -m http.server 8080 --directory "Week-1/Inkwell"*

### B.2 Week 2 — User Management REST API

*cd Week-2 && npm install && echo PORT=5000 > .env && npm run dev*

Import User Management REST API.postman_collection.json into Postman.

### B.3 Week 3 — Database Integration System

*cd Week-3 && npm install && cp .env.example .env*

Set MONGODB_URL, then: npm run dev. Import postman/collection.json; hit GET /api/health first.

## Annexure C — Sample API Envelopes

*POST /api/users { "name": "John Doe", "email": "john@example.com", "age": 25 }*

*POST /api/tasks { "title": "Complete project", "description": "Finish DB system", "completed": false, "user": "<ObjectId>" }*

## Annexure D — Weekly Work Log (Summary)

| Week | Focus | Major Activities |
|---|---|---|
| Week 1 | Frontend | Design system, layout, JS interactivity, a11y audit, README |
| Week 2 | REST API | Express layering, validators, CRUD, Postman, error middleware |
| Week 3 | Database | Schemas, relationships, pagination, populate, cascade delete |
| Post | Report + Advanced | Formal report; advanced backend blueprint (auth, ops, security) |

## Annexure E — Declaration of Non-Modification of Milestone Code

This report and its accompanying Internship-Report/ directory were produced without altering the source files under Week-1/, Week-2/, or Week-3/. Advanced backend work is presented as a designed extension blueprint in Chapter 6 for evaluation of additional engineering depth.

---

**— End of Report —**

*Shiv Pratap Singh · DecodeLabs Full Stack Internship · Batch 2026*